

2-3 · MongoDB 구축 및 비정형 로그 적재 · 실습 1.5H

비정형 로그 데이터 적재와 집계 조회

1,000건을 넣고 집계하고 인덱스 전후를 재 본다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① `.venv` 가상환경에 `pymongo` 를 설치하고 접속한다
- ② `load_logs.py` 로 접속 로그 1,000건을 `insert_many` 한다
- ③ `$match·$group·$sort` 로 레벨별·경로별·분 단위 집계를 만든다
- ④ 인덱스 생성 후의 `totalDocsExamined` 를 직접 재서 비교한다

재 보지 않으면 효과를 알 수 없습니다 — 인덱스는 만들기 전에 먼저 재고, 만든 뒤 다시 쥘니다

다섯 단계와 미니 과제

적재는 파이썬, 집계와 인덱스는 mongosh 에서 — 확인 포인트마다 숫자를 맞춰 봅니다

1 환경

.venv
pymongo

2 적재

load_logs.py
1,000건

3 집계

레벨별
경로별

4 시계열

\$dateToString
분 단위

5 인덱스

explain
전후 비교

1단계

파이썬 환경 — 가상환경부터

1 `mkdir -p ~/data-eng/week2 · cd ~/data-eng/week2`
1주차에 만든 폴더 구조를 이어 쓴다

2 `python3 -m venv .venv · source .venv/bin/activate`
프로젝트마다 라이브러리를 따로 두는 장치 — 프롬프트 앞에 (.venv) 가 붙는다

3 `pip install pymongo`
`python3 -c "import pymongo; print(pymongo.version)"` 으로 버전이 나오면 준비 완료

나가기 · 다시 들어오기

deactivate

source ...activate

WSL2·macOS·리눅스 모두 같은 명령 — 앞으로 파이썬 실습마다 씁니다

2단계

load_logs.py 가 하는 일



levels = ['INFO','INFO','INFO','WARN','ERROR'] — INFO 가 자주 나오게 가중 · status 는 ERROR 면 500

python3 load_logs.py → inserted: 1000 · mongosh 의 countDocuments() 는 1003 (2챕터에서 남은 3건 + 1,000건)

두 번 실행하면 2,000건 — DB 가 중복을 막지 않는다

왜 막지 않나

_id 를 직접 지정하지 않아 중복 검사가 없다

- 스크립트는 실행할 때마다 새 문서 1,000건을 만든다
- 자동 _id 는 매번 새 값 — 같은 내용이라도 다른 문서

VS

되돌리기 · 막기

다시 적재하려면 먼저 비운다

- mongosh: `db.access_logs.deleteMany({})`
- 실무는 자연키에 유니크 인덱스 — 중복은 DB 가 막게

집계 다섯 가지 — 파이프라인과 확인 포인트

하려는 일	파이프라인	확인
레벨별 건수	<code>\$group _id: '\$level' → \$sort</code>	INFO 가 가장 많다(600 전후)
ERROR 경로 순위	<code>\$match ERROR → \$group '\$path' → \$sort</code>	errors 내림차순
두 값으로 묶기	<code>_id: { level: '\$level', path: '\$path' } → \$limit 5</code>	조합 상위 5
경로별 평균	<code>\$avg: '\$status' · \$sum: 1</code>	ERROR 섞이면 200 보다 크다
분 단위 시계열	<code>\$dateToString '%Y-%m-%d %H:%M' · \$cond</code>	10행 · cnt 와 errors 함께

match → group → sort 를 SQL 의 WHERE → GROUP BY → ORDER BY 와 대응해 읽는다 · \$cond 는 CASE WHEN

인덱스 — 재 보고 판단한다

1 만들기 전 — explain('executionStats')
totalDocsExamined 가 약 1,003 — 전체 문서를 훑는다

2 createIndex({ level: 1, ts: -1 })
getIndexes() 로 생겼는지 확인 · 1=오름차순, -1=내림차순

3 만든 뒤 — 다시 측정
ERROR 건수(200 안팎)만 훑으면 성공 — 줄지 않았다면 인덱스를 안 탄 것, 필드 이름을 다시

복합 인덱스 규칙

왼쪽부터
이어 쓴 만큼만

{ level: 1, ts: -1 } 은 level 로 거르고 ts 로 정렬하는 쿼리엔 맞지만 ts 만으로 거르는 쿼리엔 쓰이지 않는다 — MySQL 인덱스도 같은 규칙

미니 과제 — 세 가지

사용자별 상위 5명

01

\$group _id: '\$user'
\$sort → \$limit 5

에러 코드별 건수

02

\$match ERROR → _id:
'\$error.code'
중첩 필드는 문자열로

\$match 위치 비교

03

맨 앞 vs \$group 뒤
explain 결과 차이

상위 5명이 각 20건 안팎, 에러 코드별 _id 가 null 이 아닌 값으로 나오면 성공

여기까지 됐으면 성공

- 1 프롬프트 앞에 (.venv) · pymongo 버전 출력
- 2 inserted: 1000 · countDocuments() 가 1003
- 3 레벨별 INFO 최다 · ERROR 경로 순위 내림차순
- 4 분 단위 10행이 시간 순, cnt 와 errors 가 함께
- 5 totalDocsExamined 약 1,003 → 200 안팎

숫자가 안 맞으면 필드 이름부터 — 스키마가 없어 오타를 DB 가 잡아 주지 않습니다

2-3 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

mongosh 명령을 SQL 과 나란히 정리하고, 2-4 에 쓸 컨테이너를 확인합니다

✓ mongosh ↔ SQL 대응표

✓ 오늘의 체크리스트

✓ 자주 겪는 오류 네 가지

✓ 2-4 ERD 설계 예고

가져올 것

실습 중 막혔던 지점 메모